

P1181R0

Proposing `unless`

Draft Proposal, 2018-09-08

This version:

<https://wg21.link/P1181R0>

Author:

[Morwenn](#)

Audience:

EWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Source:

github.com/Morwenn/if-not-proposal/blob/master/P1181.bs

Abstract

We propose `if not (condition)` and a few consistent additions to the C++ language, mostly as a convenience feature, but also because it might help to avoid some subtle classes of problems. The goal is rather simple: applying a negation to the condition passed to an `if not` statement *after* having applied the contextual conversion of the condition to `bool`.

Table of Contents

1 Revision History

1.1 Revision 0

2 Motivation

2.1 Declaring a variable with `if`

2.2 Avoiding tricky overloads of `operator!`

3 Clarifications & Extensions

3.1 `while not`

3.2 `if constexpr`

References

Informative References

§ 1. Revision History

§ 1.1. Revision 0

Initial release.

§ 2. Motivation

Introducing `if not` is mainly a matter of convenience: for most purposes `if (not condition)` is enough to not take a branch when `condition` evaluates to `true`. The main idea is to lift the negation into the boolean domain since that is the domain conditional statements typically deal with.

Moreover there are a few corner cases where it is not possible to negate the `if` condition naively without a workaround, which this proposal intends to make easier to deal with. Consider the examples in the following subsections.

§ 2.1. Declaring a variable with `if`

When `if` is used to declare a variable, there was no easy and clean way to negate the condition before C++17:

```

// C++14, best effort, might be confusing
if (auto opt = some_optional()); else {
    // Do stuff
}

// C++17, separate declaration and condition
if (auto opt = some_optional() ; not opt) {
    // Do stuff
}

```

With the proposed extensions, the condition above could be written as follows:

```

if not (auto opt = some_optional()) {
    // Do stuff
}

```

We believe that the notation itself is clear enough and should be rather obvious to everyone reading the conditional statement.

§ 2.2. Avoiding tricky overloads of `operator!`

If `operator!` is overloaded, `if (not condition)` is not guaranteed to have a result contextually convertible to `bool`, which might result in tricky corner cases when writing generic algorithms.

Consider the following naive implementation of `std::find_if_not`:

```

template<typename InputIterator, typename Predicate>
InputIterator find_if_not(InputIterator first, InputIterator last, Predicate pred)
{
    for (; first != last; ++first) {
        // Fails to compile when operator! doesn't return a
        // result contextually convertible to bool
        if (!pred(*first)) {
            break;
        }
    }
    return first;
}

```

If `operator!` is overloaded in such a way that it doesn't return a value contextually convertible to `bool`, the algorithm above will fail to compile, despite `pred` not violating the *Predicate* requirements (the issue came up while reviewing the inclusion of the Ranges TS in C++20, see also [\[LWG2114\]](#)). We propose that `if not` applies the negation *after* the contextual conversion to `bool`, avoiding this subtle class of problems altogether:

```
template<typename InputIterator, typename Predicate>
InputIterator find_if_not(InputIterator first, InputIterator last, Predicate pred)
{
    for (; first != last; ++first) {
        // Safe even when operator! is overloaded: the contextual
        // conversion to bool is applied before the negation
        if not (pred(*first)) {
            break;
        }
    }
    return first;
}
```

The current workaround is to use `static_cast<bool>(pred(*first))` before `!` is called which, as far as I know, is not what standard library implementations of algorithms do. Using `if not` would be an idiomatic solution to avoid this kind of problem and produce more correct generic code without extra effort. It would ensure that the negation always happens in the boolean domain.

§ 3. Clarifications & Extensions

First of all, while I use `not` through the proposal because of personal preference, it is intended that `if !(condition)` works too. Second, `if not` should also negate the condition even in the presence of an *init-statement* as introduced in C++17.

§ 3.1. `while not`

The control flow statements `if` and `while` tend to evolve in a somewhat symmetrical manner, we propose to allow `while` and `do while` to benefit from the same augmentation in a similar manner.

§ 3.2. `if constexpr`

It would be logical to allow the same extension `if constexpr` to benefit from the same extension. The form `if constexpr not` would be preferred to `if not constexpr` because it subjectively reads better when spelled with an exclamation mark. On the other hand [P1073] introduces a `constexpr!` token which would probably be preferred by max munch rule unless special cased if one tried to write `if constexpr! (condition)`, which might be seen as a drawback. This will probably need some bikeshed.

§ References

§ Informative References

[LWG2114]

Daniel Krügler. [Incorrect contextually convertible to bool requirements](https://wg21.link/LWG2114). 09 December 2011. URL: <https://wg21.link/LWG2114>

[P1073]

Richard Smith; Andrew Sutton; Daveed Vandevoorde. [constexpr! functions](https://wg21.link/P1073R1). 22 June 2018. URL: <https://wg21.link/P1073R1>